Password Strength Checker

A simple Python project that tells you how safe your password is

Project type	Command-line tool (runs in the terminal)
Language	Python 3
Library used	re (built-in regular expressions)
Main file	password_checker.py
Level	Beginner friendly

1. About This Project

This project is a small program that checks how strong a password is. You type a password, and the program instantly tells you if it is **Weak**, **Medium** or **Strong**. If the password is not good enough, it also tells you exactly what is missing, so you know what to fix.

I made this project to practise Python basics like functions, loops, conditions and string handling. I also wanted to build something useful that people can understand and use in real life, because weak passwords are one of the most common reasons accounts get hacked.

2. Why Password Strength Matters

Many people use passwords like *123456* or *password*. These can be guessed in seconds. A strong password mixes different types of characters and is long enough that it is very hard to guess. This tool helps people learn what a good password looks like.

3. What the Program Can Do

- Checks the password length (minimum 8 characters).
- Checks for at least one number (digit).
- Checks for at least one capital letter and one small letter.
- Checks for at least one special character like ! @ # $ %.
- Shows a clear message about what is missing.
- Keeps asking for passwords again and again until the user types **exit**.

4. How the Checking Works

The program checks the rules one by one, in order. As soon as one rule fails, it stops and shows the message for that rule. If everything passes, the password is called Strong.

Step	What is checked	Result if it fails
1	Password has at least 8 characters	Weak
2	Has at least one digit (0-9)	Weak
3	Has at least one uppercase letter (A-Z)	Weak
4	Has at least one lowercase letter (a-z)	Weak
5	Has at least one special character	Medium
6	All the rules above are passed	Strong

Why is the last check called Medium? A password with letters, numbers and good length is already decent, so if only the special character is missing, I felt "Medium" was a fair label instead of "Weak".

5. Code Explanation (in Simple Words)

The whole program has two functions. Here is what each part does.

The `re` module

This is Python's built-in tool for pattern matching. I use it only once, to find out if the password contains any special character.

`check_password_strength(password)`

This is the brain of the project. It receives a password and returns a message. Inside it:

- `len(password) < 8` counts the characters.
- `char.isdigit()`, `isupper()` and `islower()` look at each character and say if it is a number, capital letter or small letter.
- `any(...)` becomes True if at least one character matches. With `not` in front, we find out when nothing matches.
- `re.search(...)` looks for a special character from the list `! @ # $ % ^ & ( ) < > . ?`

`password_checker()`

This is the part the user actually sees. It prints a welcome message, then runs a `while True` loop so the program never closes by itself. Each time, it asks for a password, sends it to the checking function and prints the result. If the user types **exit** (in any letter case), the loop stops with `break` and a goodbye message is shown.

`if __name__ == "__main__":`

This line means: run the program only when this file is opened directly. It is a good habit that keeps the code clean if the file is imported somewhere else later.

6. Full Source Code

This is the complete code of the project. Save it as `password_checker.py`.

```
import re

def check_password_strength(password):
    if len(password) < 8:
        return "Weak: Password must be at least 8 characters long."

    if not any(char.isdigit() for char in password):
        return "Weak: Password must contain at least one digit."

    if not any(char.isupper() for char in password):
        return "Weak: Password must contain an uppercase letter."

    if not any(char.islower() for char in password):
        return "Weak: Password must contain a lowercase letter."

    if not re.search(r'[!@#$%^&()<>.?]', password):
        return "Medium: Password must contain a special character."

    return "Strong: Your password is strong and secure!"


def password_checker():
    print("Welcome to the Password Strength Checker")

    while True:
        password = input("Enter your password (or type 'exit' to quit): ")

        if password.lower() == 'exit':
            print("Thank you for using this tool! Goodbye!")
            break

        result = check_password_strength(password)
        print(result)


if __name__ == "__main__":
    password_checker()
```

7. How to Run the Project

- Install Python 3 from python.org if you do not have it.
- Save the code in a file named `password_checker.py`.
- Open a terminal in the same folder (in VS Code, press `Ctrl + ``).
- Run the command: `python password_checker.py`
- Type any password and press Enter. Type **exit** to close the program.

Important note for VS Code users: run the file in the **Terminal**, not in the Output panel. The Output panel of the Code Runner extension cannot take keyboard input, so typing "exit" will not work there. Use the option *Run Python File in Terminal* instead.

8. Sample Test Cases

These results come from running the actual code:

Password typed	Output shown
abc	Weak: Password must be at least 8 characters long.
abcdefgh	Weak: Password must contain at least one digit.
abcdefg1	Weak: Password must contain an uppercase letter.
ABCDEFG1!	Weak: Password must contain a lowercase letter.
Abcdefg1	Medium: Password must contain a special character.
Abcdef1!	Strong: Your password is strong and secure!

9. Limitations (Being Honest)

This is a learning project, so it is not perfect. Here are the things I know can be better:

- It only checks rules. A password like *Password1!* passes every rule but is still easy to guess.
- It does not check against lists of common leaked passwords.
- The password is visible on the screen while typing.
- It stops at the first problem, so the user sees only one issue at a time.

10. Future Improvements

- Hide the typed password using Python's `getpass` module.
- Show all missing rules together instead of one at a time.
- Add a score out of 100 or a strength bar.
- Block very common passwords like 123456 and qwerty.
- Build a simple window (GUI) using Tkinter or a web version using Flask.

11. Conclusion

This project helped me understand how functions, loops, conditions and regular expressions work together in a real program. It is small, but it solves a real problem and it is easy to extend. I plan to keep improving it with the ideas listed above.